Qwen

From EEG to Multi-Modal: A Schema-First Blueprint for Scalable, Reproducible BCI in Entertainment Systems

This report provides a comprehensive analysis of the strategic framework for expanding Horror$Place's canonical signal processing pipeline to support additional physiological modalities while ensuring cross-platform consistency and offline reproducibility. The core objective is to generalize the existing EEG-centric infrastructure into a unified, schema-first system capable of safely and scalably incorporating new biosignals. This approach prioritizes deterministic experiment replay and leverages AI-assisted development under strict adherence to schema compliance and safety guardrails before evaluating the adaptive logic that steers user experience. The analysis synthesizes the provided research goals, conversation history, and supplementary context to construct a detailed blueprint for this expansion.

Foundations of a Unified BCI Contract and Schema-Driven Development

The foundational strategy for expanding Horror$Place's BCI infrastructure is rooted in a schema-driven, or "schema-first," development methodology [3]. This paradigm elevates explicit data contracts from mere documentation to the single source of truth governing every aspect of the system, from low-level device abstraction to high-level engine integration and AI-assisted code generation. The central artifact of this approach is the Brain-Computer Interface Contract, a generalized successor to the current `EEGFeatureContractv1`. This contract serves as the immutable specification for all BCI-related data exchanged between systems, ensuring structural integrity, semantic clarity, and machine-enforceable rules. The overarching intent is to create a stable, predictable, and auditable environment where new features and modalities can be added systematically without compromising the integrity of the entire platform [9].

The proposed `BCIContract` generalizes the current EEG-focused schema by introducing a flexible, nested structure for features. Instead of hard-coding modality-specific fields like `alpha_rel` or `beta_rel`, the new schema would define a list of feature objects, each containing metadata about its origin and type. This allows for the seamless integration of diverse biosignals such as electroencephalography (EEG), functional near-infrared spectroscopy (fNIRS), electrocardiography (ECG), and galvanic skin response (GSR). An example structure would look like this: a top-level envelope containing versioning, timestamps, session IDs, and device/experiment metadata, followed by a `features` array. Each entry in this array would specify the `modality` (e.g., 'EEG', 'fNIRS'), the `type` of feature calculated (e.g., 'relative_band_power', 'hemodynamic_response', 'hrv_rmssd'), and the `value`. This modular design maintains strict schema compliance while providing the extensibility needed for future expansion. To ensure long-term compatibility and traceability, any changes to the contract would follow a formal versioning policy, preventing issues with backward or forward compatibility that can arise when upgrading schemas [9].

This schema-first approach directly informs the implementation of the Device Abstraction & Feature Layer (`Research Object 1`). The logical device registry, which maps abstract identifiers like `consumer_4ch` or `research_32ch` to physical hardware or software streams, becomes a critical component of this system. The registry file, `EEGDeviceRegistry.json`, would need to be extended to include fields specifying the sensor's native modality and its raw sample rate. For instance, a real board might have a `sampling_rate_hz` of 500, while an fNIRS stream from another device might have a rate of 19.5 Hz [17]. This metadata allows the backend drivers—a Python feature server, a C++ BrainFlow plugin, or an LSL interface—to correctly configure their internal processing pipelines. The unified `IEEGDevice` interface, implemented across C#, C++, and Rust, acts as the stable surface area that all engines and AI tools interact with, insulating them from the complexities of the underlying hardware [2]. By defining this interface and the associated device registry, the system creates a clear separation of concerns: the registry manages *what* devices are available, the drivers manage *how* to talk to them, and the feature server manages *what* features to compute.

The implementation of this framework relies heavily on generating language-specific types from the JSON Schema itself. For Python, Pydantic models or standard dataclasses would be derived from the schema, providing strong typing and automatic validation. In C# for Unity and C++ for Unreal, corresponding classes and structs would be generated to mirror the schema's structure. These schema-derived types are then used by logging helpers—such as `EEGLogger.LogSample(sample, runContext)` in C#—which are responsible for merging the feature envelope, the computed features, and the horror context before serializing the final payload to a compact JSON line. This entire process is

enforced by Continuous Integration (CI) jobs that automatically validate log samples produced during tests against the latest schema, rejecting any output that deviates from the contract. Furthermore, these CI jobs would also lint the schemas themselves, flagging any attempts to reintroduce banned fields like raw waveforms or personally identifiable information (PII), thereby enforcing privacy policies like the Dead-Ledger principle from the outset [3] [19].

The table below outlines the key components of the schema-driven development framework, detailing their purpose and the technologies involved in their implementation.

Component	Description	Technologies / Artifacts
BCI Contract Schema	The master JSON Schema defining the structure of all BCI telemetry. It includes the `envelope`, `features` array, and `horror_context`. It enforces `additionalProperties: false` and defines numeric ranges.	JSON Schema, Pydantic Models (Python), C# Classes (Unity), C++ Structs (Unreal)
Device Registry	A JSON file (`EEGDeviceRegistry.json`) mapping logical device IDs to their backend implementations (BrainFlow board ID, LSL descriptor) and metadata (channels, sample rate, modality).	`logical_id`, `backend`, `board_id`, `channels_eeg`, `sampling_rate_hz`, `modality_type`
Unified Device Interface	A language-agnostic interface (`IEEGDevice`) defining the methods `Configure`, `Start`, `Stop`, and `GetLatestFeatures()`. All device drivers must conform to this.	C#, C++, Rust Trait
Feature Server	A Python service that consumes data from registered devices via BrainFlow or LSL, computes features based on the canonical pipeline spec, and outputs validated `BCIContract` objects over TCP or LSL.	BrainFlow `BoardShim` and `DataFilter`, Python, TCP sockets, LSL
Engine Provider Interfaces	Minimal, device-agnostic interfaces exposed to gameplay code in Unity (`IEEGFeatureProvider`) and Unreal (`UEEGFeatureSubsystem`) to deliver processed features.	Unity C#, Unreal C++ Subsystems
Logging & Validation Helpers	Functions or classes that merge all data into a single payload and use the schema to validate and serialize it to NDJSON format.	`write_ndjson_line()` (Python), `EEGLogger.LogSample()` (C#)

By establishing this rigorous, schema-driven foundation, Horror$Place creates a system where change is deliberate, verifiable, and safe. Developers and AI assistants are guided by a clear set of rules, reducing the cognitive load and minimizing the risk of introducing subtle bugs or inconsistencies. This disciplined approach transforms BCI development from an ad-hoc engineering problem into a repeatable, scalable research process, laying the essential groundwork for the successful integration of multi-modal physiological data.

Cross-Platform Consistency Through Payload Standardization

Ensuring consistent behavior across different game engines, specifically Unity and Unreal, is a critical requirement for the expanded BCI infrastructure. However, the research goal intentionally scopes this challenge away from complex, engine-specific adaptive logic and toward a much narrower, more manageable objective: achieving parity at the BCI contract layer. The directive is not to make the scare logic identical in both engines, but rather to guarantee that when presented with the same payload, both engines will interpret and react to it identically. This is achieved by creating a well-defined, unambiguous boundary at the BCI contract level, making downstream engine-specific work "mechanical rather than research-intensive." This strategy effectively offloads the difficult problem of cross-engine behavioral alignment to the simpler task of ensuring data format uniformity.

The primary mechanism for enforcing this consistency is the strict definition and universal adoption of the `BCIContract` schema across the entire pipeline. Every component—from the Python feature server to the C++ and C# backends in Unity and Unreal—must produce and consume data that conforms precisely to this schema. The contract dictates the exact format of the telemetry summary, the `bci-payload` itself, and even the structured proof-of-seed data like `bcistate` and `bciseed`. By standardizing these elements, the system ensures that the information flowing from the BCI hardware abstraction layer is semantically identical regardless of the engine receiving it. The engine integrations do not see the raw, unprocessed data; they only ever interact with the standardized feature structs, such as the Unity `struct EEGFeatures` and the Unreal `USTRUCT(BlueprintType) FEEGFeatures`. Both of these structures are generated from the same underlying schema definition, guaranteeing that their memory layout and field definitions are aligned, thus eliminating a major class of potential errors related to data deserialization and interpretation [23].

The implementation of this strategy is realized through a series of well-defined, decoupled layers. At the base, the device abstraction and feature server layer, as described in `Research Object 1`, is responsible for generating the canonical feature data. This layer uses a fixed processing pipeline specified in `EEGCanonicalV1` (or its generalized multi-modal equivalent) to compute features like relative band powers, composite scores, and signal quality flags [12]. The output of this server is a stream of NDJSON objects, each one being a self-contained, schema-valid `BCIContract` payload. This payload contains all necessary information, including device metadata, timestamps, and the complete feature vector.

On top of this, the engine integration layer provides a thin, abstracted API. In Unity, this is the singleton `EEGFeatureService` implementing the `IEEGFeatureProvider` interface, while in Unreal, it is the `UEEGFeatureSubsystem`. These services act as intermediaries, connecting the game logic to the backend data source. Crucially, they are agnostic to whether the data is coming from a live Python feature server or from a replay of a previously recorded NDJSON log. The replay functionality, a cornerstone of the project's focus on reproducibility, is implemented by a specialized replay source that also satisfies the `IEEGFeatureProvider` or `UEEGFeatureSubsystem` interface. When the engine is in replay mode, it simply plugs this replay source into the director instead of the live provider. Because both sources expose the exact same public methods (`HasFeatures`, `GetEEGFeatures`), the rest of the engine—including the `HorrorDirector`—is completely unaware of the difference. This elegant solution ensures that any evaluation of adaptive strategies is performed under perfectly controlled, identical conditions in both engines, validating that the logic is indeed mechanical and portable.

The table below details the key components and their roles in ensuring cross-platform consistency at the BCI contract layer.

Component	Role in Ensuring Consistency	Technology	Interaction
BCI Contract Schema	Defines the exact structure of the payload, ensuring all platforms parse the same fields in the same way.	JSON Schema	Used by all platforms to generate typed objects and validate incoming/outgoing data.
NDJSON Telemetry Stream	Provides a simple, line-based format for streaming payload data that is easy to parse and replay reliably.	Text File (NDJSON)	The data format passed between the feature server and the engines.
Engine Provider Interfaces	Abstracts the data source behind a common interface (`IEEGFeatureProvider`, `UEEGFeatureSubsystem`), hiding implementation details.	C# Interfaces, Unreal Subsystems	The sole point of contact for gameplay code to access BCI features.
Replay Source	Implements the same provider interface as the live source, allowing the engine to consume recorded data as if it were live.	Custom C# Class / C++ Class	Plugs into the director in place of the live provider for testing and analysis.
Schema-Backed Feature Structs	Generates identical `EEGFeatures` structs in Unity (C#) and Unreal (C++) from the same schema, ensuring no mismatch.	Code Generation from Schema	The in-memory representation of the payload within each engine.

This layered architecture successfully isolates the complexity of BCI data acquisition and processing from the creative, engine-specific implementation of adaptive experiences. The heavy lifting of ensuring data fidelity and computational consistency is done once in the shared, schema-driven layers. Once a `HorrorDirector` strategy is developed and proven in one engine, porting it to the other becomes a matter of wiring it up to the common provider interface, a significantly less complex and risky endeavor. This

approach embodies the principle of separating what is known (the data contract) from how it is used (engine-specific logic), enabling rapid iteration and scaling across multiple platforms with confidence in the underlying data integrity.

Prioritizing Deterministic Reproducibility via Experiment Replay

A core tenet of the research goal is the explicit prioritization of offline reproducibility and deterministic experiment replay over raw real-time performance. This philosophical choice elevates the project from a simple BCI integration into a rigorous scientific research platform. The ability to precisely reproduce experiments, compare results across different sessions and labs, and validate the effectiveness of adaptive logic is considered paramount. This is achieved through a meticulously designed system built around tiered NDJSON telemetry streams and a robust replay mechanism that makes live and replayed data indistinguishable to the engine.

The foundation of this system is the creation of a standardized, PII-free telemetry format. All logged data is structured as a separate JSON object on each line of a text file, adhering to the NDJSON (newline-delimited JSON) format. This choice is ideal for both streaming data capture and subsequent batch processing. The data captured is organized into several distinct streams, mirroring the `bci-telemetry-raw/redacted/summary` and `bci-events-stream` concepts mentioned in the research goal. The `raw` stream captures the initial output from the feature server, while the `redacted` stream may contain sanitized versions suitable for broader sharing. The `summary` stream provides a condensed view of key metrics, and the `events` stream logs discrete occurrences like the start or end of a block or condition. Each object within these streams is an instance of the `BCIContract`, enriched with metadata about the session, experiment, and run, ensuring that every data point is fully contextualized [13] [18].

The cornerstone of the reproducibility framework is the experiment orchestration and replay system (`Research Object 6`). Experiments, defined as collections of conditions, blocks, scenes, and configurations, are themselves declared as data in a `ExperimentConfig` schema. A Python orchestrator script reads this configuration, handles randomization, and instructs the target engine (Unity or Unreal) which specific run to execute. During execution, the engine's `ExperimentManager` logs critical metadata, such as `RunStart` and `RunEnd` events, into the NDJSON telemetry stream,

embedding the relevant block and condition IDs into the `BCIContract` envelopes. This creates a complete, self-contained record of the entire experimental session.

The power of this system lies in its replay capability. A `ReplayConfig` specifies the path to an NDJSON log file, a speed factor, and optional filters to replay only specific portions of an experiment. The implementation involves a custom replay source that implements the same `IEEGFeatureProvider` interface used for live data. This replay source first parses the entire NDJSON log file, building an ordered sequence of `(timestamp, EEGFeatures)` pairs. During the replay, it feeds these features into the engine's provider subsystem at the correct times, simulating a live session with perfect fidelity. Because the engine's gameplay systems, including the `HorrorDirector`, only call methods like `GetEEGFeatures()`, they cannot distinguish between data coming from a live server or from the replay buffer. This deterministic nature is crucial for reliable comparison of different seeds, adapters, and director strategies. Researchers can run the same experiment dozens of times, always starting from the exact same input data, and measure the variance in outcomes attributable solely to the logic within the engine itself.

To ground this system in reality and ensure the underlying feature extraction is accurate and consistent, the plan calls for the generation of "test vectors" (`Research Object 3`). These are small, synthetic datasets designed to produce known, expected outputs from the canonical signal processing pipeline. For example, a test vector could consist of a sine wave at a specific frequency to verify band power calculations, or a simulated stressor event to check the response of composite scores. The outputs of the reference Python implementation on these vectors are stored as ground-truth values. A critical part of the CI pipeline is a job that runs the C++ and any AI-generated feature extraction code on these same test vectors and asserts that the numeric outputs agree with the expected values within a predefined tolerance. This automated numerical validation ensures that the core logic is deterministic and correct across all platforms, forming the bedrock of the entire reproducible science effort.

The table below summarizes the key telemetry streams and their purposes, illustrating the systematic approach to data capture for reproducibility.

Telemetry Stream Name	Purpose	Contents Example	Key Characteristics
bci-telemetry-raw	Captures the full, unmodified output from the feature server immediately after computation.	Full `BCIContract` object with all features, signals, and context.	High fidelity, potentially large size.
bci-telemetry-redacted	Contains a sanitized version of the telemetry, suitable for sharing or archival.	`BCIContract` with PII or sensitive raw data fields removed.	Privacy-preserving, lower fidelity than raw.
bci-telemetry-summary	Provides a condensed, aggregated view of the session for quick overview and high-level analysis.	Aggregated stats per block/condition (e.g., mean stress score, total time in zone).	Compact, optimized for dashboards and summary reports.
bci-events-stream	Logs discrete, significant events that occurred during the experiment.	`{ "event_type": "RunStart", "run_id": "exp_A_cond_1_run_3", "timestamp": ... }`	Time-stamped, categorical, marks boundaries of experimental units.

By building a system that is fundamentally designed for deterministic replay, Horror$Place gains a powerful tool for iterative development and scientific validation. It allows for the isolation and measurement of variables, the debugging of adaptive logic, and the accumulation of a verifiable corpus of experimental data that can be analyzed and re-analyzed over time.

The Role of AI-Assisted Development as a Constrained Generator

The integration of AI-assisted development into the BCI infrastructure project is not envisioned as an autonomous creative partner but as a highly constrained, rule-following generator. The goal is to leverage the efficiency of Large Language Models (LLMs) to accelerate the implementation of well-defined patterns and boilerplate code, while simultaneously mitigating the risks associated with their unpredictable nature [5]. This is achieved through a sophisticated two-stage validation workflow where AI-generated artifacts must first pass a battery of automated, non-negotiable checks before their primary functional correctness is even evaluated. This approach treats the AI as a powerful template-filling engine, with its output rigorously vetted by the system's schema, safety, and numerical integrity requirements.

The first and most critical stage of this workflow is the pre-validation gate. Before any AI-generated code or configuration is merged into the main codebase, it must pass a series of automated checks orchestrated through the CI/CD pipeline. These checks form a hard filter that rejects non-compliant outputs immediately. The first gate is **Schema**

Compliance. Any AI proposal for a new device in the registry, a new mapping in the director config, or a change to a telemetry schema must produce output that validates against the corresponding JSON Schema. The CI job will fail if the AI introduces a new field, misnames an existing one, or violates a data type or range constraint. This ensures that all artifacts adhere to the single source of truth defined by the `BCIContract`.

The second gate is **Safety Guardrail Adherence**. For any proposed `HorrorMappingConfig`, the CI must validate that it conforms to the `HorrorSafetyConfig` schema. This means checking that parameters like maximum tension increase rates, cooldown durations, and minimum signal quality thresholds fall within acceptable, pre-defined ranges. The system explicitly enforces guardrails that prevent harmful or unethical mappings, such as those that could induce excessive stress or operate on poor-quality data [4]. Proposals that violate these hard-coded safety policies are rejected out of hand.

The third gate is **Numerical Fidelity**, which applies primarily to the signal processing and feature extraction components. As outlined in `Research Object 3`, the system relies on a suite of synthetic test vectors with known, expected outputs. When the AI proposes an implementation of the canonical pipeline in C++ or another language, the CI runs this new code against the test vectors. It then compares the program's output against the ground-truth values. If the results do not match within a specified tolerance, the build fails. This ensures that the core algorithmic logic remains deterministic and accurate, a prerequisite for offline reproducibility.

Finally, the fourth gate is **Policy and Governance Alignment**. This involves verifying that all new artifacts declare the appropriate Tier (e.g., `standard`, `mature`, `research`) and that any high-intensity or experimental feature properly references the required Dead-Ledger surfaces and policies. This aligns the AI's output with the project's overarching governance model, treating safety and ethics as non-optional attributes of any BCI-related component [3].

Only after an AI-generated artifact has successfully passed all these stringent pre-validation gates is it considered "contract-correct" and "entitlement-safe." At this point, it proceeds to the second stage of evaluation: assessing its functional contribution to the adaptive logic. Here, the AI's proposals are judged on their ability to steer entertainment metrics (UEC, ARR, CDL) more effectively. Researchers can propose new mapping curves, PID controller parameters, or piecewise strategies in the `HorrorMappingConfig`. These proposals are tested offline by having them process large batches of recorded NDJSON logs. The resulting output is then fed into analysis notebooks that measure key performance indicators, such as the amount of time the player spends in a desired tension

band or the avoidance of prolonged stress overload. Only proposals that demonstrate improved steering of these metrics, while operating entirely within the verified safety envelopes established by the first stage of validation, are promoted to production. This two-stage process ensures that creativity and optimization are pursued only within a tightly controlled and safe sandbox, preventing bad ideas from ever reaching the evaluation stage and safeguarding the integrity of the entire system.

The following table outlines the AI-assisted development workflow, highlighting the constraints and feedback loops that guide the AI's contributions.

Stage	Gate / Check	Description	Technology / Method
Pre-Validation	Schema Compliance	Ensures all generated code/configs strictly adhere to the official JSON Schemas (`BCIContract`, `HorrorMappingConfig`, etc.).	Schema validators integrated into CI jobs.
Pre-Validation	Safety Guardrail Adherence	Verifies that proposed mappings respect hard-coded safety limits on tension, duration, and signal quality.	Rule-based validation against `HororSafetyConfig` schema.
Pre-Validation	Numerical Fidelity	Confirms that the AI's implementation of the canonical pipeline produces the exact same results as the reference on synthetic test vectors.	Automated unit tests running on curated test vector datasets.
Pre-Validation	Policy & Governance Alignment	Checks that new artifacts correctly declare their tier and comply with Dead-Ledger governance requirements.	CI checks for mandatory metadata fields and policy references.
Post-Validation	Adaptive Logic Evaluation	Assesses the functional impact of passing proposals on steering entertainment metrics (UEC, ARR, CDL) in offline replay environments.	Offline analysis using NDJSON logs and custom evaluation notebooks.

This disciplined workflow transforms the AI from a potential liability into a powerful accelerator. By forcing the AI to learn and operate within a clearly defined set of constraints, the system harnesses its capabilities for repetitive tasks while maintaining rigorous control over quality, safety, and consistency.

Integrating New Physiological Modalities Beyond EEG

Expanding the canonical signal processing pipeline to support additional physiological modalities is the central technical challenge of this research initiative. While the current infrastructure is centered on EEG, the overarching design principles of schema abstraction, canonical processing, and tiered telemetry provide a robust framework for this expansion. The successful integration of new biosignals like functional near-infrared spectroscopy (fNIRS), electrocardiography (ECG), or respiration (Resp) hinges on the

careful generalization of existing components to handle heterogeneous data types, sampling rates, and feature extraction methodologies.

The first step in this process is the evolution of the canonical signal processing specification. Currently defined by `EEGCanonicalV1` as a JSON file dictating windowing, filtering, and band power calculations [12], this specification must be extended to become a multi-modal processing blueprint. This generalized spec would need to define separate processing chains or stages for each connected modality. For instance, it could specify that an 'ECG' stream undergoes a 0.5-30 Hz bandpass filter and has heart-rate variability (HRV) features like RMSSD extracted, while an 'fNIRS' stream is processed with a low-pass filter to model the slow hemodynamic response and has HbO/HbR concentration changes calculated [12]. The feature server would then read this enhanced spec to instantiate and orchestrate parallel processing pipelines for each active data source, ultimately merging their outputs into a single, coherent payload for the engines.

A more significant architectural change is the generalization of the telemetry schema itself. The current `EEGFeatureContractv1` must evolve into the `BCIContract`. As previously discussed, this involves moving from a flat object structure to a more flexible, nested format. The `features` section of the contract should become an array of objects, where each object describes a single measurement. This object would contain fields for `modality` ('EEG', 'fNIRS', 'ECG'), `feature_type` ('relative_band_power', 'hemodynamic_response', 'hrv_metric'), and the `value`. This structure elegantly accommodates the diverse vocabulary of features across different physiological domains. For example, it could represent an EEG alpha power, an fNIRS HbO change, and an ECG RR interval all within the same telemetry packet, providing a rich, multi-modal feature vector for the `HorrorDirector` to consume.

One of the most significant technical hurdles in multi-modal integration is handling data streams with vastly different sampling frequencies. For example, a typical EEG system might record at 1000 Hz, while an fNIRS system operates at ~8 Hz, and an ECG at 256 Hz [12] [17]. Simply trying to process these streams together at a single rate is inefficient and can lead to aliasing or loss of temporal precision. The system must implement a synchronization strategy. One viable approach is to perform all processing independently on each stream's native timeline and then interpolate or resample the resulting low-frequency features to a common, lower timestamp resolution (e.g., 10 Hz) for the final payload. Another approach is to trigger processing events based on a common clock or external stimulus markers present in the data, ensuring that features from different modalities are aligned relative to a shared event, such as the start of a game encounter. Adopting standards from the neuroscience community, such as the Brain Imaging Data

Structure (BIDS), which recommends storing each modality in separate files with clear metadata about its sampling frequency and starting time, would provide a robust foundation for managing this complexity [13] [18].

The literature on multi-modal fusion offers valuable insights for leveraging this combined data. Research shows that feature-level fusion, where features from different modalities are concatenated and fed into a classifier, can yield significant improvements over using any single modality alone [14]. For instance, a study combining EEG and fNIRS for ALS classification achieved 85.38% accuracy, a substantial improvement over EEG alone (73.23%) or fNIRS alone (61.56%) [14]. This suggests that the `BCIContract`'s flexible feature array is already perfectly suited for this type of analysis. Further research has even demonstrated the ability to predict fNIRS signals from EEG, finding that higher-frequency EEG bands (gamma, beta) are dominant predictors of hemodynamic responses [17]. This has profound implications for the `HorrorDirector`: it opens the possibility of inferring states from harder-to-measure modalities using proxies from easier-to-acquire ones, potentially simplifying hardware requirements for certain applications. The `HorrorDirector` could be configured to use gamma band power as a proxy for a deep brain state typically measured by fNIRS, if analysis shows a strong correlation with the desired entertainment metric.

The following table contrasts the requirements for EEG with those of potential new modalities, highlighting the necessary extensions to the core infrastructure.

Modality	Typical Use Case	Signal Characteristics	Processing Requirements	Derived Features
EEG	Cortical activity, attention, emotion	Microvolt-scale electrical potentials, high temporal resolution (up to 1kHz+), susceptible to motion/artifact.	Bandpass filtering (e.g., 1-40Hz), notch filtering (50/60Hz), PSD estimation (Welch's method).	Relative band powers (delta, theta, alpha, beta, gamma), composite scores (calm, stress).
fNIRS	Hemodynamic response, localized cortical oxygenation.	Near-infrared light absorption, low frequency (<1Hz), millisecond-level temporal resolution.	Modified Beer-Lambert law, low-pass filtering (e.g., 0.01-0.1Hz).	Concentration changes of Oxygenated Hemoglobin (HbO) and Deoxygenated Hemoglobin (HbR).
ECG	Cardiac activity, autonomic nervous system (HRV).	Millivolt-scale electrical potential, regular peaks (QRS complexes), sampling rate ~256Hz.	R-peak detection, RR-interval calculation, bandpass filtering (e.g., 0.5-35Hz).	Heart Rate (HR), Root Mean Square of Successive Differences (RMSSD), Low-Frequency/High-Frequency (LF/HF) ratios.
Respiration (Resp)	Breathing rate, depth, irregularity.	Capacitance or airflow signal, very low frequency (<0.5Hz), variable amplitude.	Filtering (e.g., 0.1-0.8Hz), peak/trough detection.	Breath rate (brpm), breath amplitude, inter-breath interval.

By systematically addressing these challenges—from extending the processing spec and generalizing the telemetry schema to solving the problems of synchronization and

exploring fusion techniques—Horror$Place can build a truly multi-modal BCI platform. This will unlock a richer, more nuanced understanding of the player's state, paving the way for more sophisticated and personalized adaptive experiences.

Strategic Synthesis and Implementation Roadmap

The expansion of Horror$Place's BCI infrastructure represents a mature, research-driven engineering effort aimed at building a scalable, verifiable, and safe platform for multi-modal physiological computing. The synthesis of the provided materials reveals a coherent and powerful strategy built upon four core pillars: a schema-first approach to ensure structural integrity; a relentless focus on offline reproducibility to enable rigorous scientific validation; a constrained AI-assisted workflow to balance speed with safety; and a tiered governance model to enforce ethical and safety policies. This strategic direction moves beyond simple signal processing to construct a foundational research platform where experimentation, iteration, and innovation can occur within a controlled and predictable environment.

The central insight of this strategy is the elevation of the `BCIContract` schema to the status of the ultimate authority. By making the schema the single source of truth for all data, the system achieves a high degree of decoupling between its components. The device abstraction layer, the feature processing logic, the engine integrations, and the AI-assisted code generators all derive their behavior from this central document. This design choice is what enables the prioritization of deterministic experiment replay; because the data format is fixed and unambiguous, replaying a session from a captured NDJSON log becomes a trivial mechanical process, allowing researchers to isolate and measure the effects of their adaptive logic with unprecedented precision. The explicit narrowing of the cross-platform consistency goal to the BCI contract layer further exemplifies this philosophy. By ensuring that both Unity and Unreal engines receive and interpret the exact same payload, the difficult problem of cross-engine behavioral alignment is sidestepped, leaving the creative work of adaptive design to be ported mechanically.

The implementation roadmap can be structured chronologically, progressing from foundational schema work to practical application and analysis. The immediate priority is the generalization of `EEGFeatureContractv1` into a true `BCIContract` and the extension of the `ExperimentConfig` schema. This involves defining the flexible, nested-object structure for features and creating a comprehensive suite of synthetic test vectors for the new modalities to be supported. Concurrently, the canonical processing

specification (`EEGCanonicalV1`) must be evolved into a multi-modal processing blueprint. With these core schemas defined, the next phase involves implementing the backend components: developing the C++ and Python feature processing modules that can handle the diverse streams, and creating the replay source that can feed data from NDJSON logs to the engine providers.

Once the core infrastructure is in place, the focus shifts to practical application and analysis. This involves training the AI-chat agents on the new schema and the constrained development playbook, beginning with the implementation of drivers for a new, single modality (e.g., fNIRS). The `HorrorDirector` can then be updated to consume features from this new modality, with its mappings being validated first against the new safety and schema rules, and then through offline replay analysis. Finally, the team can begin to explore multi-modal fusion, using the rich, concatenated feature vectors from the `BCIContract` to build more sophisticated predictive models of player state and engagement, guided by the analysis notebooks and subject to the same rigorous validation and governance procedures. This phased, incremental approach ensures that each step builds upon a solid foundation, allowing the system to scale gracefully while maintaining the high standards of safety, consistency, and scientific rigor that define the project's vision.

Reference

1. Summary of over Fifty Years with Brain-Computer Interfaces ... https://pmc.ncbi.nlm.nih.gov/articles/PMC7824107/
2. Brain-Computer Interfaces, Open-Source, and ... https://www.frontiersin.org/journals/computer-science/articles/10.3389/fcomp.2021.661300/full
3. The protection of mental privacy in the area of neuroscience https://www.europarl.europa.eu/RegData/etudes/STUD/2024/757807/EPRS_STU(2024)757807_EN.pdf
4. Computer Science https://arxiv.org/list/cs/new
5. Language Models Are Capable of Metacognitive Monitoring and ... https://arxiv.org/html/2505.13763v2
6. Guy Wilson - Tacit | LinkedIn https://www.linkedin.com/in/guy-wilson
7. Sensors, Volume 21, Issue 2 (January-2 2021) – 353 articles https://www.mdpi.com/1424-8220/21/2

8. 333333 23135851162 the 13151942776 of 12997637966 ftp://ftp.cs.princeton.edu/pub/cs226/autocomplete/words-333333.txt
9. Backward and forward compatibility with Avro schema https://stackoverflow.com/questions/67766981/backward-and-forward-compatibility-with-avro-schema
10. Detecting Biothreat Agents: From Current Diagnostics to ... https://pubs.acs.org/doi/10.1021/acssensors.8b00420
11. 2017 TECHNICAL SUMMARIES https://www.researchgate.net/profile/Sylvia-Bardet/publication/313360488_Novel_molecular-based_fluorescent_nanoparticles_for_three-photon_excited_microscopy_at_1700_nm/links/5eb997eda6fdcc1f1dd2c2af/Novel-molecular-based-fluorescent-nanoparticles-for-three-photon-excited-microscopy-at-1700-nm.pdf
12. A multi-modal dataset of electroencephalography and ... - PMC https://pmc.ncbi.nlm.nih.gov/articles/PMC12144169/
13. Physiological recordings - Brain Imaging Data Structure 1.11.1 https://bids-specification.readthedocs.io/en/stable/modality-specific-files/physiological-recordings.html
14. Multimodal fusion of EEG-fNIRS: a mutual information-based ... https://pmc.ncbi.nlm.nih.gov/articles/PMC7984774/
15. IntelliJ IDEA 2023.3 (233.11799.241 build) Release Notes https://youtrack.jetbrains.com/articles/IDEA-A-2100661779/IntelliJ-IDEA-2023.3-233.11799.241-build-Release-Notes
16. [PDF] hyperledger-fabricdocs Documentation https://hlf.readthedocs.io/_/downloads/en/release-1.0/pdf/
17. Multimodal Autoencoder Predicts fNIRS Resting State ... - PMC https://pmc.ncbi.nlm.nih.gov/articles/PMC9547786/
18. Physiological and other continuous recordings https://bids-specification.readthedocs.io/en/v1.5.0/04-modality-specific-files/06-physiological-and-other-continuous-recordings.html
19. [PDF] Practical Internet of Things Security - ResearchGate https://www.researchgate.net/profile/Ali_Al-Qurabat2/post/Can_any_one_share_me_a_Book_of_Practical_Internet_of_Things_Security/attachment/5a415ea84cde266d587da2a7/AS%3A575551220207619%401514233512763/download/Brian+Russell%2C+Drew+Van+Duren-Practical+Internet+of+Things+Security-Packt+Publishing+%282016%29.pdf
20. Appl. Sci., Volume 15, Issue 10 (May-2 2025) – 570 articles - MDPI https://www.mdpi.com/2076-3417/15/10

21. 10.1007@978 981 15 5566 4 | PDF | Smart Grid - Scribd https://www.scribd.com/document/497925779/10-1007-978-981-15-5566-4
22. Dicionario portugues https://www.academia.edu/32592435/Dicionario_portugues
23. indonesian-idf.txt https://www.researchgate.net/profile/Bowo-Prasetyo/publication/273141354_indonesian-idf/data/54f9dbd10cf25371374ffccb/indonesian-idf.txt?origin=publication_list